

Aula 7 — Interrupções e ruído de contato

O mecanismo, e a arquitetura que ele obriga

Microcontroladores — DENE/UFMT

OBJETIVOS

- Descrever a interrupção como uma chamada de sub-rotina forçada pelo hardware, em termos do ciclo de busca-execução do encontro 2.
- Identificar as variáveis compartilhadas entre o tratamento e o programa principal, e proteger as que precisam de `volatile` ou de atomicidade.
- Estimar latência e flutuação de latência, e decidir quais tarefas toleram cada uma.
- Explicar o repique de contato como fenômeno mecânico, e distinguir as soluções de hardware das de software.
- Implementar confirmação por tempo dentro do tratamento de um temporizador, sem bloquear o programa principal.
- Distinguir os dois níveis de prioridade do PIC18 e dizer o que o hardware salva sozinho e o que o compilador precisa salvar.
- Orçar o tratamento contra o intervalo da própria fonte, e justificar por que trabalho lento não pode morar nele.
- Estruturar um programa como tratamento curto mais trabalho diferido, e reconhecer nisso o escalonador que o laço principal já é.

1 Cinco dívidas com a mesma forma

Encontro	O problema	Custo
3	A tela bloqueia enquanto o lá de 440 Hz precisa de uma inversão a cada 1,14 ms	1,3 ms
4	O programa espera o conversor sem ter o que fazer	60 ciclos
5	O recarregamento do Timer0 atrasa o intervalo seguinte	acumulativo
5	Estouros são perdidos se o laço demorar mais que o intervalo	silencioso
5	O CCP não alcança 440 Hz; o software alcança e bloqueia	—

Cinco problemas, uma forma só: **o processador precisa aparecer num instante determinado, e a única maneira que ele tinha era ir olhar.**

A Q4 da integradora pediu a descrição de um mecanismo — desvio assíncrono, independente do que estivesse rodando, com retorno ao ponto de origem. Ele existe, está no sílício desde sempre, e este encontro é sobre ele.

2 O que é uma interrupção

Uma interrupção é uma **chamada de sub-rotina que o hardware faz sem pedir licença**. Quando a condição ocorre, ao final da instrução corrente o processador empilha o contador de programa, carrega nele um endereço fixo e continua executando dali. A instrução RETFIE desempilha e a execução volta exatamente para onde estava.

CONCEITO

Não há mágica nenhuma nisso, e o encontro 2 já tinha todas as peças: o PC é um registrador, e carregar outro valor nele é o que qualquer desvio faz. A novidade é apenas **quem** decide carregar.

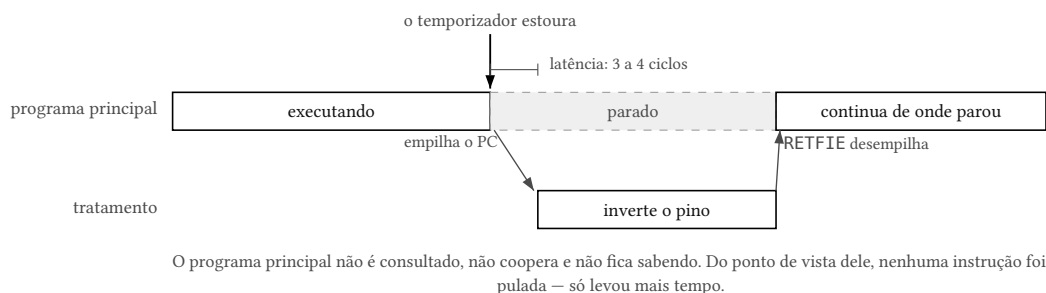


Figura 1: O programa principal não é consultado. Do ponto de vista dele, nenhuma instrução foi pulada — o programa apenas levou mais tempo para chegar ao fim.

O PIC18 tem dois endereços de desvio: 0x0008 para interrupções de alta prioridade e 0x0018 para as de baixa. São endereços fixos em silício, como o 0x0000 do reset.

E aqui volta o princípio do encontro 0 pela quarta vez. O bootloader ocupa o começo da Flash, **incluindo os endereços 0x0008 e 0x0018**. Quem responde à interrupção primeiro é ele, não a sua aplicação.

NESTE KIT

O bootloader precisa então reencaminhar o desvio para um endereço mais alto, onde o seu tratamento realmente está, e o arquivo de ligação da aplicação precisa concordar com esse endereço. É a mesma classe de erro do simulador que começava a executar no menor endereço do HEX: quando não bate, nada acusa — o tratamento simplesmente nunca roda.

Verificar antes do R7: qual deslocamento de vetor o bootloader do XM118 usa, e se o projeto de exemplo do curso já traz o arquivo de ligação correspondente.

2.1 Latência

Entre a condição ocorrer e a primeira instrução do tratamento executar passam-se alguns ciclos: terminar a instrução corrente, empilhar, carregar o PC. São três a quatro ciclos — menos de um microssegundo a 16 MHz.

O número que costuma importar não é a latência: é a **variação** dela.

ATENÇÃO

A latência mínima é fixa. A máxima depende do que estava rodando: se uma instrução de dois ciclos acabou de começar, espera-se mais um; se outro tratamento estiver em curso, espera-se ele terminar.

Para inverter um pino a 440 Hz, uma flutuação de um microssegundo sobre 1136 μ s é irrelevante — 0,09%, ou menos de dois centésimos de semitom. Para medir a largura de um pulso, a mesma flutuação é o erro da medida.

3 O que o tratamento não pode estragar

O tratamento roda no meio do programa principal, sem que ele saiba. Duas coisas que funcionavam param de funcionar.

3.1 *volatile*

O compilador não sabe que o tratamento existe. Para ele, uma variável que o laço lê e nunca escreve é uma constante — e otimizar a leitura para fora do laço é correto.

CONCEITO

```
uint8_t pronto = 0;           /* escrito pelo tratamento */

while (!pronto) { }          /* laço infinito: o compilador
                              leu 'pronto' uma vez so */
```

`volatile` é a declaração de que o valor pode mudar por fora do fluxo visível, e obriga a releitura a cada acesso. Sem ela o programa funciona em depuração, com otimização desligada, e trava com otimização ligada.

Este é um sintoma de bancada de primeira grandeza: **funciona no modo de depuração e trava no modo normal**. Quando isso acontecer, a primeira suspeita é uma variável compartilhada sem `volatile`.

NOTA

3.2 Atomicidade

`volatile` resolve a visibilidade, não a **indivisibilidade**.

CONCEITO

Um contador de dezesseis bits é lido em duas instruções, um byte de cada vez. Se a interrupção cair entre as duas, os dois bytes vêm de valores diferentes — lê-se um número que nunca existiu.

Com o contador em 0x00FF passando a 0x0100, uma leitura infeliz devolve 0x01FF: mais alto que os dois. Num relógio de milissegundos, isso é um salto de 256 ms para o futuro, uma vez a cada 256 incrementos.

```
volatile uint16_t ms = 0;

uint16_t ms_ler(void)
{
    uint16_t a, b;
    do {
        a = ms;
        b = ms;
    } while (a != b);    /* le duas vezes; aceita quando concordam */
    return a;
}
```

A alternativa é desabilitar a interrupção durante a leitura, o que é mais direto e tem um preço: a interrupção fica atrasada por esses poucos ciclos. Ler duas vezes até concordar não atrasa ninguém e custa uma releitura rara.

NOTA

Qual das duas usar depende de qual recurso está mais apertado — tempo de resposta ou ciclos. Essa pergunta reaparece o resto do semestre.

3.3 Reentrância

O XC8 não gera funções reentrantes por padrão. Os parâmetros e as variáveis locais de uma função ficam num espaço **fixo**, escolhido em tempo de compilação — não numa pilha.

CONCEITO

A consequência é direta: se o programa principal está no meio de `formatar()` e o tratamento chama `formatar()` também, a segunda chamada escreve por cima das variáveis locais da primeira. Quando o principal retomar, elas terão mudado.

O compilador detecta boa parte desses casos e avisa, ou duplica a função em duas cópias, uma para cada contexto. Duplicar gasta Flash e resolve; o aviso, quando vem, não deve ser silenciado.

ATENÇÃO

Mas há um caso em que ninguém avisa: funções de biblioteca. Chamar rotina de divisão, de ponto flutuante ou de formatação dos dois lados é entrar exatamente nesse defeito, e ele se manifesta como valor errado **raro**, no principal, sem relação visível com o botão que o causou.

A regra prática que o resto do curso vai seguir: o tratamento não chama nada que o principal também chame. Se precisar, duplique de propósito, com outro nome.

4 Prioridade, e o que custa entrar

O PIC18 tem **dois** níveis. O bit IPEN, em RCON, decide se eles existem: com IPEN = 0 o chip opera em modo de compatibilidade e tudo desvia para 0x0008; com IPEN = 1, cada fonte tem seu bit de prioridade, alta desvia para 0x0008 e baixa para 0x0018.

O que dois níveis compram é **uma** coisa: a fonte de alta prioridade pode interromper o tratamento da de baixa. CONCEITO

Sem isso, um tratamento longo atrasa todos os outros — e a flutuação de latência que a seção anterior discutiu passa a ser do tamanho do pior tratamento do programa, não do pior caso do hardware.

Com isso, o buzzer de 440 Hz pode ser atendido enquanto o tratamento do botão ainda está rodando. O preço é que agora existem duas fronteiras de dados, e não uma: variável compartilhada entre os dois tratamentos precisa do mesmo cuidado que uma compartilhada com o principal.

Entrar num tratamento não é de graça, e vale saber o que se paga.

Salvo pelo hardware	WREG, STATUS e BSR, na pilha rápida de um nível
Salvo pelo compilador	Todo o resto que o tratamento tocar: registradores de trabalho, ponteiros, temporários
Restaurado	RETFIE FAST devolve os três em uma instrução; o resto sai da RAM

A pilha rápida tem **um** nível só. Se o tratamento de baixa prioridade a usar e um de alta prioridade ocorrer no meio, o segundo sobrescreve o conteúdo do primeiro. ATENÇÃO

Por isso o compilador só confia nela no nível alto, e salva o contexto do nível baixo em RAM. É a razão pela qual um tratamento de baixa prioridade tem prólogo maior — e, se ele tocar muitas variáveis, esse prólogo pode passar do próprio trabalho útil.

Tratamento curto não é elegância: é o que mantém o prólogo proporcional.

5 O ruído de contato

Uma chave mecânica não fecha uma vez: as lâminas se chocam, ricocheteiam e estabilizam. Durante alguns milissegundos o pino oscila entre os dois níveis.

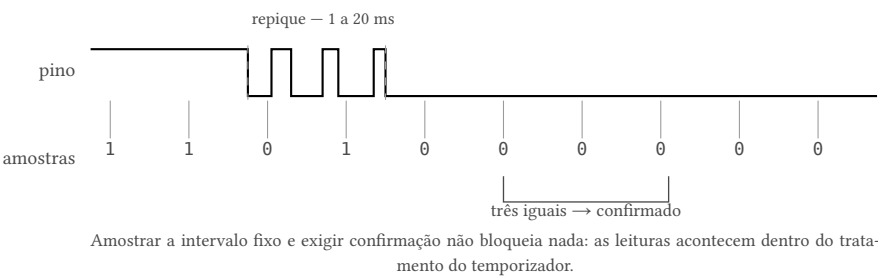


Figura 2: O repique é físico, dura de 1 a 20 ms e é característico **daquela** chave. A duração é medida, não copiada da internet.

Ligar um botão a uma entrada de interrupção sem tratamento é a receita para o defeito mais confuso da bancada: **um toque gera vinte eventos**. ATENÇÃO

O setpoint sobe cinco graus de uma vez, o contador pula, o menu atravessa três telas. E o número de eventos muda a cada toque, porque o repique é aleatório — o que torna o defeito quase impossível de reproduzir de propósito.

No PICSimLab o botão é ideal: fecha uma vez e pronto. O simulador **mente** aqui, e saber onde ele mente é parte do conteúdo. DIVERGÊNCIA

Repique se estuda na bancada, com osciloscópio. O **algoritmo** que o filtra, esse sim, se estuda no simulador — injetando uma sequência de amostras e verificando o que o filtro decide. Cada ferramenta para a metade que ela faz bem.

5.1 Duas famílias de solução

	Hardware	Software
Como	RC na entrada mais disparador Schmitt	Confirmação por tempo
Custo	Dois componentes por botão	Algumas linhas, uma vez
Escala	Ruim: dezesseis botões, trinta e dois componentes	Boa: o mesmo código serve para todos
Quando escolher	Quando o sinal precisa estar limpo antes do pino — entrada de interrupção, de captura ou de contagem	Quando o pino é lido por amostragem

O que não fazer: detectar a borda e chamar `__delay_ms(20)`.

CONCEITO

Funciona, e desfaz tudo que este encontro construiu — bloqueia o processador por vinte milissegundos, que é quinze vezes o pior bloqueio que o curso tinha até agora. Trocar espera cega por espera cega maior não é solução.

O que fazer: amostrar o pino a intervalo fixo, dentro do tratamento do temporizador que já existe, e só aceitar a mudança depois de N leituras consecutivas iguais. Com amostragem a 5 ms e $N = 3$, a confirmação leva 15 ms e custa algumas instruções por amostra.

6 O código

```
#define PRECARGA_5MS 45536u /* 65536 - 20000: 5 ms a 250 ns */

volatile uint16_t ms5 = 0; /* tempo, em passos de 5 ms */
volatile uint8_t botao = 1; /* estado confirmado (1 = solto) */
volatile uint8_t borda = 0; /* pulso de "acabou de ser pressionado" */

void __interrupt(high_priority) tratar(void)
{
    static uint8_t candidato = 1;
    static uint8_t iguais = 0;

    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        TMR0H = (uint8_t)(PRECARGA_5MS >> 8);
        TMR0L = (uint8_t)(PRECARGA_5MS & 0xFF);

        ms5++;

        uint8_t agora = PORTBbits.RB0; /* ADCON1 = 0x0F, encontro 4 */

        if (agora != candidato) {
            candidato = agora;
            iguais = 1;
        } else if (iguais < 3) {
            iguais++;
            if (iguais == 3 && candidato != botao) {
                botao = candidato;
                if (botao == 0) {
                    borda = 1; /* pressionado, confirmado */
                }
            }
        }
    }
}
```

```

    }
  }
}

```

Três detalhes que valem a leitura atenta.

NOTA

O tratamento **não** espera, não escreve no display e não converte nada. Ele lê um pino, mexe em três variáveis e sai. Tudo que é lento continua no programa principal.

`candidato` e `iguais` são `static` e não são `volatile`: só o tratamento os enxerga. `volatile` é para o que atravessa a fronteira, e marcar tudo como `volatile` desperdiça otimização sem comprar segurança.

`borda` é um pulso que o programa principal consome — lê e zera. É a forma mais simples de um sinal de mão única, e a que menos exige do laço principal.

A melodia, agora. Com o tratamento invertendo o pino do buzzer, o lá de 440 Hz precisa de meio período de 1136,4 μ s, ou 4545 ciclos. Pré-carga: $65536 - 4545 = 60991$.

CONCEITO

O erro fica em 4545 contra 4545,45 ciclos — 0,01%, ou cerca de um terço de centésimo de semitom, inaudível.

E, o que importa mais: a nota continua correta **durante** a atualização de tela. O bloqueio de 1,3 ms do display não impede a interrupção de acontecer; ele apenas adia o programa principal, que não é quem gera a nota.

7 A arquitetura que isto obriga

O tratamento da seção anterior lê um pino e mexe em três variáveis. Isso não foi estilo: foi orçamento.

Quanto tempo um tratamento pode durar? Menos que o intervalo da própria fonte, com folga para os outros.

CONCEITO

O Timer0 dispara a cada 5 ms — 20000 ciclos. Parece muito. Mas se o buzzer do encontro 5 estiver no mesmo programa, ele precisa de uma inversão a cada 1136 μ s, ou **4545 ciclos**. Esse é o número que manda, e ele é o menor intervalo entre dois eventos que o programa precisa atender.

Um tratamento que gastasse 6000 ciclos não estouraria o Timer0 e ainda assim perderia notas do buzzer. O orçamento não é o do evento mais frequente que o tratamento atende: é o do evento mais frequente que **existe**.

Daí sai a regra que organiza o resto do semestre:

No tratamento	Ler um pino ou um registrador, rearmar o temporizador, incrementar um contador, levantar um sinalizador
---------------	---

No principal	Escrever no display, converter para texto, dividir, formatar, transmitir, decidir
--------------	---

Repare no que está do lado direito: **tudo que este curso já mediu e achou caro**.

ATENÇÃO

A tela custa 5200 ciclos, a conversão 60 mais o software que a cerca, a divisão por dez é chamada de biblioteca. Nenhum desses cabe num orçamento de 4545 ciclos dividido com as outras fontes — e, pior, nenhum deles tem custo **constante**.

Um tratamento de duração variável estraga a única coisa que a interrupção veio oferecer, que é aparecer na hora certa.

O mecanismo que liga os dois lados é o sinalizador, e ele já está no código:

```
/* no tratamento: 3 ciclos */
borda = 1;

/* no laço principal: 5200 ciclos, e tudo bem */
if (borda) {
    borda = 0;
    setpoint++;
    lcd_mostrar(setpoint);
}
```

Esta é a estrutura profissional de firmware, e ela tem nome: **tratamento curto e trabalho diferido**.

CONCEITO

O tratamento não resolve o problema. Ele apenas registra que o problema existe, num lugar que o programa principal vai olhar, e sai. Quem resolve é o principal, quando puder — e “quando puder” é aceitável porque o instante crítico, o que não podia ser perdido, já foi atendido.

Foi isso que o curso vinha tentando fazer desde o encontro 3, sem ter a ferramenta: separar **quando uma coisa precisa acontecer** de **quanto tempo ela leva**.

Há um custo, e ele é de projeto, não de ciclos.

DIVERGÊNCIA

O laço principal agora precisa girar rápido o bastante para consumir os sinalizadores antes que outro evento chegue. Se ele demorar 300 ms escrevendo uma tela inteira e o usuário apertar o botão duas vezes nesse intervalo, o segundo toque some — *borda* já estava em 1.

Um `uint8_t` como sinalizador guarda “aconteceu”, não “aconteceu duas vezes”. Para guardar quantidade, é preciso um contador; para guardar ordem e conteúdo, uma fila. E é exatamente aqui que o desenho começa a pedir um sistema operacional.

8 Para onde isto vai: tarefas

O laço principal com sinalizadores **já é um escalonador**. Um escalonador ruim, escrito à mão, mas um escalonador: ele olha uma lista de coisas pendentes, escolhe uma e executa até o fim.

CONCEITO

O que ele não tem é preempção, prioridade e bloqueio. Uma tarefa longa não pode ser suspensa no meio para outra mais urgente rodar, e não há como uma parte do programa dizer “me acorde quando isto chegar” — só há como perguntar de novo na volta seguinte.

Um sistema operacional de tempo real é o que se ganha ao tornar essas três coisas explícitas. O desenho não muda; o que muda é quem cuida dele:

	Laço com sinalizadores	Com tarefas
Trabalho diferido	Trecho dentro do laço	Tarefa, com pilha própria
Sinalizador	<code>volatile uint8_t</code>	Semáforo ou fila
Esperar	Perguntar de novo na volta	Bloquear, sem gastar processador
Urgência	A ordem em que você escreveu os <code>if</code>	Prioridade, com preempção
Evento perdido	Sinalizador já em 1	Fila com profundidade e transbordo visível

```
/* o mesmo desenho, agora com tarefas */
void ISR_botao(void)
{
```

```

    BaseType_t acordou = pdFALSE;
    xSemaphoreGiveFromISR(sem_botao, &acordou); /* so sinaliza */
    portYIELD_FROM_ISR(acordou);
}

void tarefa_interface(void *p)
{
    for (;;) {
        xSemaphoreTake(sem_botao, portMAX_DELAY); /* bloqueia, nao pergunta */
        setpoint++;
        lcd_mostrar(setpoint);                    /* 5200 ciclos, a vontade */
    }
}

```

Repare no sufixo `FromISR`. Ele existe porque as duas metades deste encontro continuam valendo: dentro do tratamento não se pode bloquear, e a função precisa ser segura para ser chamada de um contexto que interrompeu qualquer outro.

NOTA

Um RTOS não revoga nada do que foi visto aqui — *volatile*, atomicidade, reentrância, orçamento de tempo. Ele dá nome às soluções e as implementa uma vez, bem feitas, em vez de cada projeto reinventá-las.

E, honestamente, ele nem sempre compensa.

DIVERGÊNCIA

Cada tarefa quer pilha própria, e RAM é o recurso mais escasso deste chip: o PIC18F4550 tem 2 KB. Quatro tarefas com 256 bytes de pilha cada consomem metade disso antes de qualquer variável do programa. O escalonador também custa ciclos a cada troca de contexto e a cada tique.

Para um termostato com três coisas a fazer, o laço com sinalizadores é a resposta certa — é menor, é previsível e cabe na cabeça. A pergunta que decide não é qual é mais moderno; é quantas atividades independentes existem e quantas delas precisam esperar por coisas diferentes ao mesmo tempo.

Este curso fica no laço, de propósito, e com o desenho certo. Quem entender a estrutura aqui troca de ferramenta sem trocar de ideia.

9 Previsão para o R7

PREVISÃO

P1. Você vai ligar o osciloscópio no botão e capturar um toque. Quantos milissegundos de repique espera ver? E quantas transições?

P2. Antes de filtrar, você vai contar eventos por toque num contador simples. Escreva quantos espera. Depois toque dez vezes: o número se repete?

P3. Com amostragem a 5 ms e confirmação de 3, qual é o atraso entre encostar no botão e o programa reagir? Esse atraso é perceptível?

P4. Se você aumentar a confirmação para 10 amostras, o que melhora e o que piora? Dê os dois números.

P5. Você vai medir o repique de dois botões diferentes da placa. Espera o mesmo número nos dois? Justifique antes de medir.

Leve esta folha preenchida e a captura de osciloscópio do P1 — ela vira a entrada do simulador na segunda metade do roteiro.

SEM NOTA

10 Exercícios

Exercício 7.1. Um programa usa `uint16_t` ms incrementado no tratamento e lido no laço principal para medir intervalos. Ele funciona, mas de vez em quando registra um intervalo absurdo, sempre positivo e sempre grande.

TAREFA

- (a) Qual é o defeito?
- (b) Por que o erro é sempre para mais, e nunca para menos?
- (c) Corrija sem desabilitar interrupções.

Exercício 7.2. Um colega resolve o repique assim: no tratamento da interrupção por mudança de estado em PORTB, chama `__delay_ms(20)` e depois lê o pino de novo.

TAREFA

- (a) Funciona?
- (b) Que dano isso causa ao resto do sistema?
- (c) Qual é a diferença conceitual entre essa solução e a da aula, além do tempo?

Exercício 7.3. Você precisa medir a largura de um pulso externo com resolução de 1 μ s, e considera fazê-lo por interrupção: anotar o tempo na borda de subida e na de descida.

TAREFA

- (a) A latência atrapalha? Justifique.
- (b) E a variação da latência?
- (c) Que recurso do chip resolveria isso melhor, e por quê?

Exercício 7.4. O tratamento roda a cada 5 ms e consome 40 ciclos. O programa principal atualiza a tela, o que custa 5200 ciclos de bloqueio.

TAREFA

- (a) Que fração do processador o tratamento consome?
- (b) Durante a atualização de tela, quantas vezes o tratamento roda?
- (c) O programa principal percebe alguma coisa?

No encontro 8: estágio de potência, atuadores e fontes de reset. Até aqui os pinos moveram LEDs e um display, que consomem miliampères. O aquecedor e o relé consomem ampères, e nenhum pino do PIC18F4550 os aciona diretamente.

NOTA

E vem junto a pergunta que a bancada faz sozinha assim que o primeiro atuador de verdade comuta: **por que ele reiniciou?**

Exercício 7.5. Um programa atende o buzzer a 440 Hz e o Timer0 a cada 5 ms. Um colega acrescenta ao tratamento do Timer0 a atualização do display, que custa 5200 ciclos.

TAREFA

- (a) O Timer0 continua sendo atendido a tempo? Justifique com números.
- (b) O que acontece com a nota do buzzer?
- (c) Onde essa atualização deveria estar, e o que precisa atravessar a fronteira para que ela funcione lá?

Exercício 7.6. O laço principal leva 300 ms para desenhar uma tela inteira. O tratamento sinaliza o botão com `volatile uint8_t` borda.

TAREFA

- (a) O usuário aperta o botão três vezes durante esse desenho. Quantos incrementos o setpoint recebe?
- (b) Troque o `uint8_t` por um contador e responda de novo.
- (c) Que informação nem o contador guarda, e que estrutura seria necessária para guardá-la?